TASK 9: Introduce Abstraction

Name: K.V.N.D.P.SAI RUPESH | Register Number: 1972260102 | Java Fundamentals and OOP - Assignment 1

Which class is abstract and which methods are abstract

Vehicle is abstract because a generic vehicle with no type has no meaningful rate. Its abstract method is ratePerHour(), since each subclass must provide a rate. Its concrete method calculateFee() is shared, because the rounding rule is the same for every vehicle. An abstract class lets the shared rule live in one place and forces each subclass to supply the part that differs.

Which capability is an interface

Chargeable is an interface, because charging is an ability that only some vehicles have. Any class (here ElectricCar) can implement it without being a Car. Pass is also an interface: a monthly pass and a day pass are different objects that share one ability, covers(entry, exit).

Why abstract class in one place and interface in another

Vehicle needs state (the regNo field) and a shared method body, which only an abstract class can provide. Chargeable and Pass need only a promise that a method exists, which is exactly what an interface does, and a class can take several of them.

How a class gets multiple behaviours

```
class ElectricCar extends Car implements Chargeable {
    public double chargingFee(int kWh) { return kWh * 12; }
}
```

ElectricCar takes behaviour from Car (through the Vehicle chain) and from Chargeable. A Java class can extend only one class, but it can implement many interfaces, so this is how it gets more than one capability.

Where a default or static method in an interface is useful

A default method gives every implementing class a standard behaviour that it may replace. For example, Pass could have default String describe() returning the pass name, which all passes share unless they override it. A static method in an interface is useful for a helper that belongs to the concept, such as a method that creates the right pass for a user type. Neither is in our code yet; they are the next step if passes grow.

Alternative: Vehicle as an interface with default methods, Chargeable as an abstract class

If Vehicle became an interface with default methods, calculateFee() could still be a default method, but the interface cannot hold the regNo field or a constructor, so each class must store its own regNo. This is more repetition, and the registration rule can no longer be enforced in one constructor. If Chargeable became an abstract class, ElectricCar could no longer extend Car and also be Chargeable, because Java has no multiple class inheritance. Time per method call is about the same in both designs, because the JVM dispatches interface and class methods with similar cost. Space per object is slightly different: the abstract-class version stores regNo once in each vehicle object, while the interface version needs the field in every implementing class. The main losses are constructors, state and multiple inheritance of classes.